The Journal of Open Source Software

NAMOSIM: A Robot Motion Planner for Navigation Among Movable Obstacles

David Brown[1 4], **Jacques Saraydaryan**[1 3 4], **Benoit Renault**[2 4], **and Olivier Simonin**[1 2 4]

1 Inria, CHROMA Team **2** INSA Lyon **3** CPE Lyon **4** CITI Laboratory

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

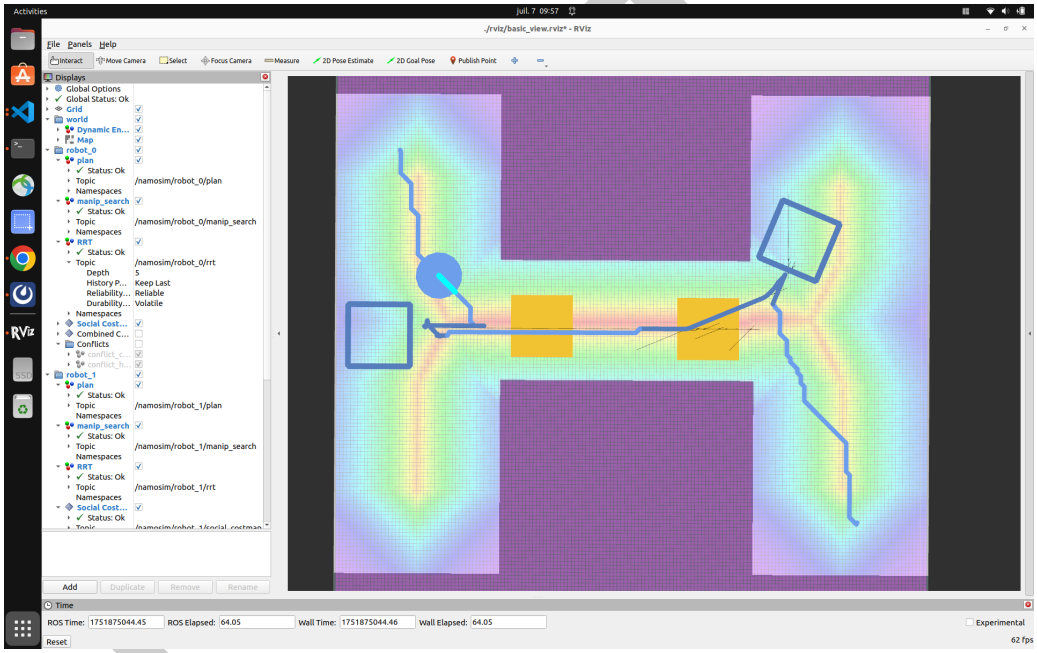

Figure 1: NAMOSIM

Summary

NAMOSIM is a mobile robot motion planner designed for the problem of **N**avigation **A**mong **M**ovable **O**bstacles (NAMO). The planner simulates robots navigating in 2D polygonal environments wherein certain obstacles can be grasped and relocated in order for the robots to reach their goals. NAMOSIM thus extends the classic navigation problem with a layer of interactivity which poses interesting research questions while remaining well-defined and amenable to both classical and learning-based approaches. NAMOSIM additionally supports multi-robot environments and provides a baseline NAMO algorithm along with a communication-free coordination strategy. The simulator includes support for holonomic and differential-drive motion models, and integrates with ROS2 for visualization in RViz.

NAMOSIM uses a modular agent-based architecture, and includes a baseline NAMO algorithm (Stilman & Kuffner, 2005) implemented in the `Stilman2005` agent, which also implements a communication-free coordination strategy for multi-robot scenarios. A variety of other agent types are implemented, and new agents utilizing alternative approaches can be created and plugged into the planner in a straightforward manner by implementing the **Agent** base class. Thus, new navigation algorithms, including those based on machine learning or AI, can be

developed within NAMOSIM, thereby facilitating reproducible research on NAMO problems.

NAMOSIM utilizes ROS2 messages for visualization of environments and plans using RViz2, and includes a number of prebuilt scenarios to use for testing and benchmarking. Scenarios are stored as SVG files, so custom scenarios can be conveniently created using a free SVG editor such as Inkscape.

NAMOSIM is packaged as a ROS2 package for easy integration into robotics projects but may also be used as a standalone Python module. The package is intended for researchers and developers working on robot navigation in dynamic environments, particularly where physical interaction is necessary.

Statement of need

Many interesting applications in autonomous mobile robotics involve physical interaction with the environment and social coordination with other agents. However, standard navigation planners assume static and non-interactive environments, limiting their usefulness in complex real-world applications. NAMO problems involve not only path planning but also introduce the need for reasoning about which obstacles to move, where to move them, and how to combine standard navigation with obstacle manipulation. NAMOSIM addresses this gap by offering a simulation environment explicitly designed to study and prototype NAMO algorithms. NAMOSIM additionally supports multi-robot environments and thus facilitates reproducible research in multi-robot navigation among movable obstacles (MR-NAMO).

Major Features

NAMOSIM provides a robust set of features to support research and development in Navigation Among Movable Obstacles (NAMO):

- **Modular Agent-Based Architecture**: The simulator is built around a flexible `Agent` interface, allowing users to implement and test custom NAMO planning algorithms. A baseline implementation of the Stilman2005 planner is included for immediate use and benchmarking.
- **Support for Multiple Robot Models**: NAMOSIM supports both holonomic and differential drive robot models, enabling realistic simulation of various robotic platforms.
- **ROS2 Integration**: Full compatibility with ROS2 allows seamless deployment on both simulated and physical robots, with built-in support for visualization in RViz2 and integration with ROS2 navigation stacks.
- **2D Environment Simulation**: The simulator provides a customizable 2D environment where users can define static and movable obstacles, supporting complex scenarios for testing navigation and manipulation strategies.
- **Extensive Testing and Documentation Utilities**: NAMOSIM includes tools for automated testing of planning algorithms and generating comprehensive documentation, facilitating reproducible research and educational use.
- **Multi-Robot Coordination**: The simulator supports multi-robot scenarios, enabling the study of implicit coordination and conflict resolution in NAMO tasks, as explored in related research (Renault et al., 2024).

These features make NAMOSIM a versatile tool for prototyping, evaluating, and deploying NAMO algorithms in diverse robotic applications.

Customizable Scenarios

NAMOSIM environments, or **scenarios**, or stored in SVG format and can be edited using any SVG editor such as Inkscape. The scenario SVG file contains the following keys elements:

- The geometry of the static map
- The polygons and orientations of all robots and movable obstacles
- Configuration settings that define the behavior the environment and robots.

The static map can also be included as an image layer inside the SVG to conveniently include ROS grid-map images generated by standard mapping tools.

Architecture

At a high-level, NAMOSIM executes a SENSE-THINK-ACT loop that performs the following functions at each iteration:

1. SENSE: Each agent senses the environment and updates its internal representation of it.
2. THINK: Each agent computes a new plan or updates its current plan.
3. ACT: Each agent selects a single discrete action to execute.

The loop is expected to execute at a regular frequency with the assumption that all agent functions are synchronized at run sequentially.

Stilman's Algorithm

NAMOSIM includes a baseline implementation of Stilman's 2005 NAMO algorithm. The key idea of this algorithm is to move obstacles in such a way as to merge disjoint components of the robot's free configuration space. The map is divided into a set of disjoint **connected components** where each cell in a given component is reachable from all the other cells in the same component. It can easily be proven that components must be separated from each other by movable obstacles or otherwise be unreachable. The algorithm functions by moving obstacles so as to join components until the robot's current component includes the goal cell.

The algorithm works by recursively performing the following two stages:

1. **SELECT_OBSTACLE_AND_COMPONENT**: The first stage performs a simplified A* grid search where the agent is allowed to pass through movable obstacles. It returns the ID of the first movable obstacle encountered on the optimal path to the goal and the ID of the component encountered after passing through the obstacle.
2. **OBSTACLE_MANIPULATION_SEARCH**: The second stage first finds a **transit path** from the robot's current position to a grasp pose near the obstacle. Then it finds a **transfer path** by performing an obstacle manipulation search to join the robot's current component to the component selected in stage 1. If this stage fails for any reason, the obstacle and component pair are added to an avoid-list and the algorithm goes back to stage 1.

The each iteration of the algorithm continues with a copy of the environment where the robot and obstacle start from the poses resulting from the end of the previous obstacle manipulation search. This algorithm is explained in greater detail in Renault's 2023 PhD Thesis (Renault, 2023).

Collision Detection

Custom agents are free to implement their own collision detection routines, however our baseline `Stilman2005` agent detects collisions using a simple binary-occupancy grid during transit paths (while not carrying an obstacle) where the robot footprint is assumed to be circular. However, when transporting a movable obstacle, the robot footprint is non-circular, and collision detection is based on the **convex swept volume** resulting from the area swept by the combined robot-obstacle footprint due to the action motion. While computationally expensive, this guarantees that all possible collisions are detected, regardless of the shape of the robot or obstacle.

Social Costmap

A novel contribution in our baseline implementation is the option to use a social costmap during the obstacle manipulation search to guide obstacle placement decisions. This allows robots to place obstacles in areas less likely to block the free passage of other agents, including humans, thus making it less likely the obstacles will need to be moved again in the future. They key heuristic of the social costmap is to *avoid narrow and center** which means the social cost is greater in narrow cooridors and at the center of open spaces. Thus helps the robot avoid placing obstacles in front of a doorway or in the center of a room. The social costmap is explained more fully in Renault (2023).

Conflict Avoidance and Deadlock Resolution

Our baseline NAMO agent also has the capability to avoid conflicts and attempt to resolve deadlocks with other agents. Conflict avoidance works by looking ahead along the agent's current plan for a fixed number of steps, called the **conflict horizon**. Within the horizon, the agent simulates each planned action and checks for a number of possible conflicts. For example, the agent may have planned to move a certain obstacle which has been moved by another robot and is no longer at the expected location. Or as another example, an action within the conflict horizon may collide with another robot that currently crossing the planned path.

The `Stilman2005` agent avoids conflicts by either pausing or planning around them. A deadlock is detected when a given conflict configuration is re-detected multiple times, even after replanning. To resolve deadlocks, the agent follows an evasion strategy as described in our IROS-2024 paper (Renault et al., 2024).

Acknowledgements

This research was supported by an Inria ADT initiative. We express our gratitude to Benoit Renault, whose PhD thesis forms the foundation of this work.

References

Renault, B. (2023). *NAvigation en milieu MOdifiable (NAMO) étendue à des contraintes sociales et multi-robots* (Theses No. 2023ISAL0105, INSA de Lyon). https://hal.science/tel-04418723

Renault, B., Saraydaryan, J., Brown, D., & Simonin, O. (2024). Multi-robot navigation among movable obstacles: Implicit coordination to deal with conflicts and deadlocks. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1–7. https://hal.science/hal-04705395

Renault, B., Saraydaryan, J., & Simonin, O. (2020). Modeling a social placement cost to extend navigation among movable obstacles (NAMO) algorithms. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 11345–11351. https://doi.org/10.1109/IROS45743.2020.9340892

Stilman, M., & Kuffner, J. J. (2005). Navigation among movable obstacles: Real-time reasoning in complex environments. *International Journal of Humanoid Robotics*, *2*(4), 479–503. https://doi.org/10.1142/S0219843605000545